Lab 03 — Azure Identity & Access Management: Zero Trust Identity Governance

Author: Emmanuel Onen

Location: Cayman Islands

Date: July 2026

Target Roles: Cloud Engineer · Identity Administrator · Security Analyst

Certifications Aligned: CompTIA Security+ · SC-300 · AZ-104 · AZ-500

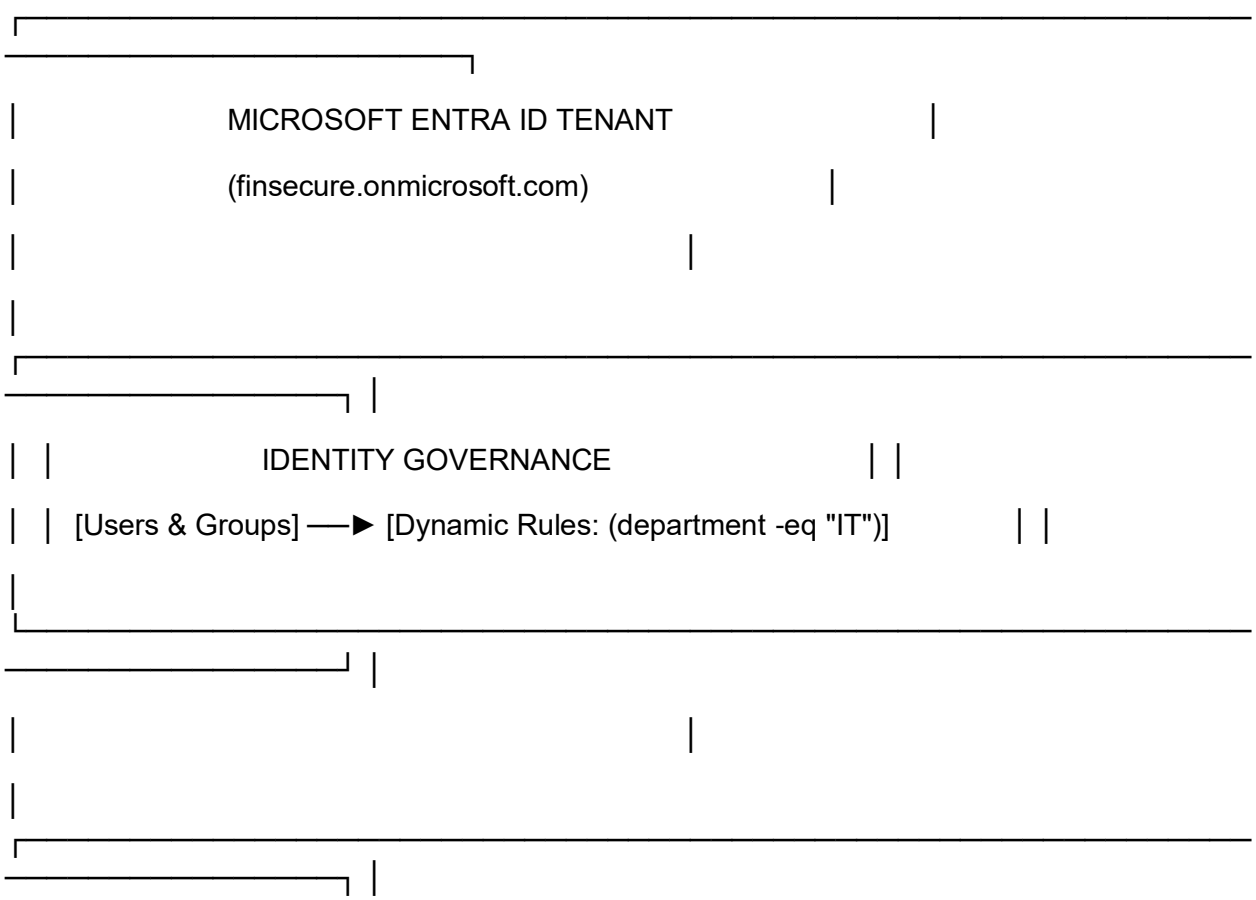
1. Executive Summary & Architecture

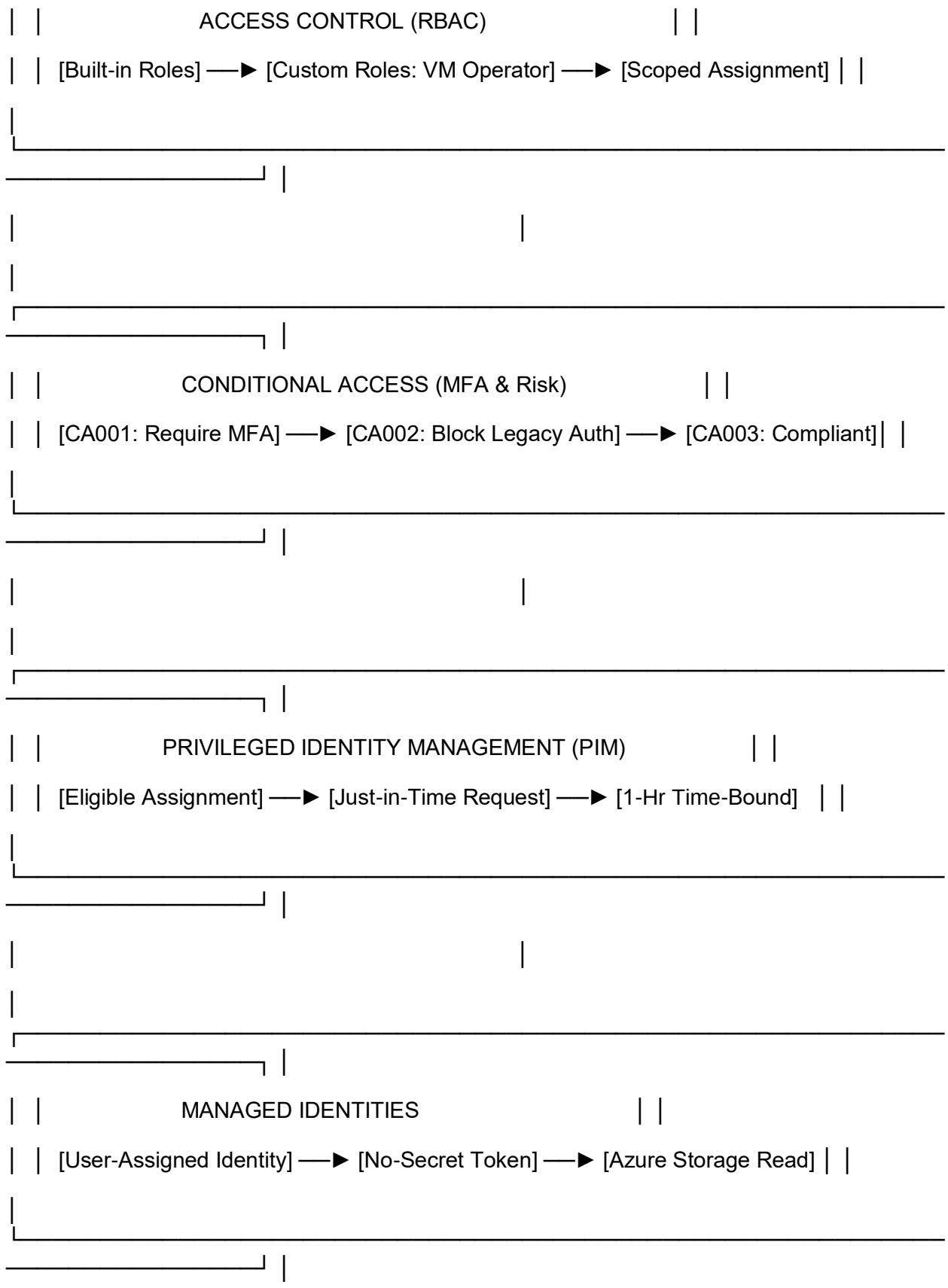
Business Scenario

FinSecure Corp, a financial services organization, is migrating infrastructure to Microsoft Azure and must comply with ISO/IEC 27001, PCI-DSS, and SOC 2 Type II governance frameworks.

To eliminate hardcoded credentials, permanent administrative privileges, and unverified access pathways, this lab implements a **Zero Trust Identity Governance Architecture** inside Microsoft Entra ID.

Identity Architecture Diagram




ACCESS CONTROL (RBAC)
[Built-in Roles] ──► [Custom Roles: VM Operator] ──► [Scoped Assignment]
CONDITIONAL ACCESS (MFA & Risk)
[CA001: Require MFA] ──► [CA002: Block Legacy Auth] ──► [CA003: Compliant]
PRIVILEGED IDENTITY MANAGEMENT (PIM)
[Eligible Assignment] ──► [Just-in-Time Request] ──► [1-Hr Time-Bound]
MANAGED IDENTITIES
[User-Assigned Identity] ──► [No-Secret Token] ──► [Azure Storage Read]

2. Step-by-Step Implementation & Proof Guide

Task 0: License Activation & Tenant Setup

UI View: Microsoft 365 Admin Center (admin.cloud.microsoft) > **Your products** displaying **Microsoft Entra ID P2 Trial** with an **Active** subscription status and 100 available licenses.

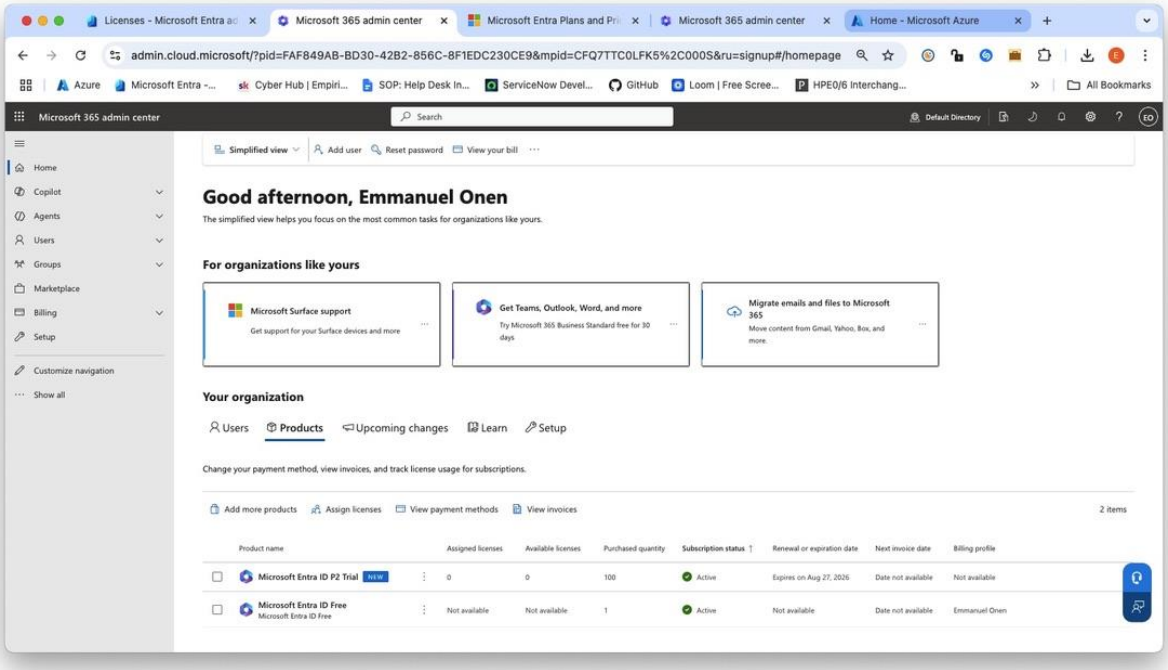


Figure 0a: Entra-p2-license-active.jpeg

Lab 3 — Azure Identity | Entra ID P2 Trial Activation | July 2026

Puporse: Activating a Microsoft Entra ID P2 license tier is a fundamental prerequisite for deploying enterprise Zero Trust governance controls. Standard Entra ID Free lacks advanced security features

Techincal Justification: Activating a Microsoft Entra ID P2 license tier is a fundamental prerequisite for deploying enterprise Zero Trust governance controls. Standard Entra ID Free lacks advanced security features; P2 capabilities are mandatory to configure Risk-based Conditional Access policies, Access Reviews, and Privileged Identity Management (PIM) for Just-In-Time (JIT) administrative escalation.

Task 1: Identity Provisioning & Dynamic Group Automation

Step 1.1: Create Test Enterprise Users

- **UI View:** Microsoft Entra Admin Center (entra.microsoft.com) > **Users** > John Doe user profile blade displaying UPN john.doe@eonenitgmail.onmicrosoft.com, Job Title Senior Cloud Engineer, and Department set to IT.

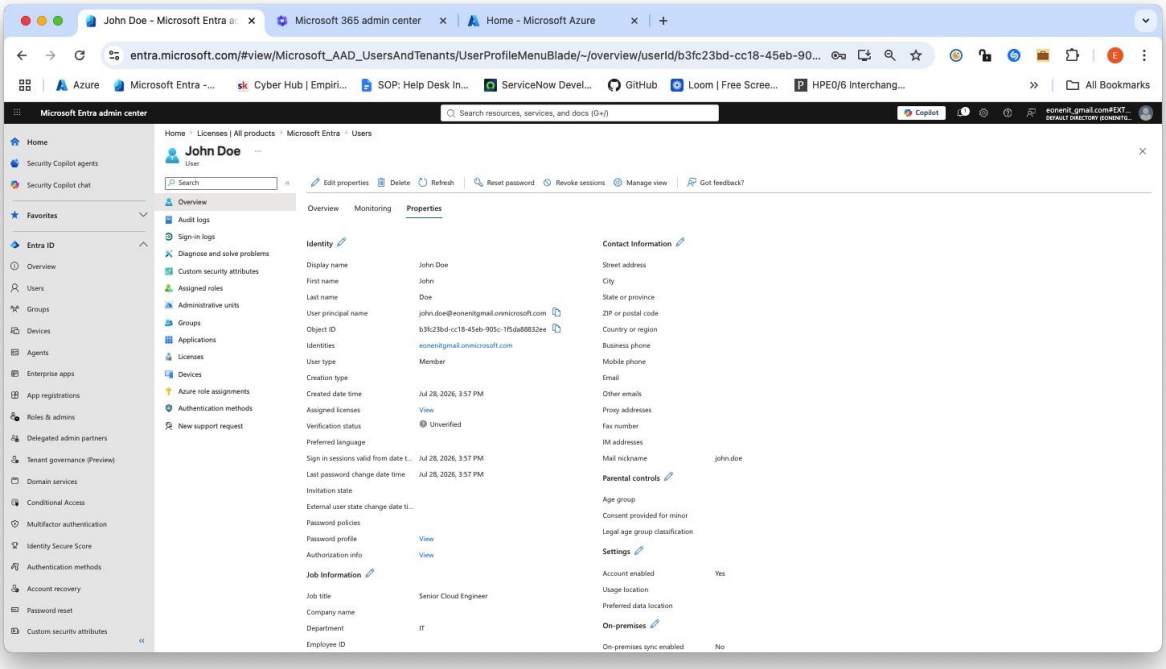


Figure 1a: -user-creation-portal.jpeg

Lab 3 — Azure Identity | User Account Provisioning | July 2026

Puporse: Provisioning targeted cloud identities with accurate metadata (such as Department and Job Title) establishes the foundation for Attribute-Based Access Control (ABAC).

Techincal Justification: Populating the IT department attribute ensures this user automatically matches dynamic group rules for automated access assignment without manual administrative overhead.

Azure Cloud Shell Workflow – Alternative to GUI

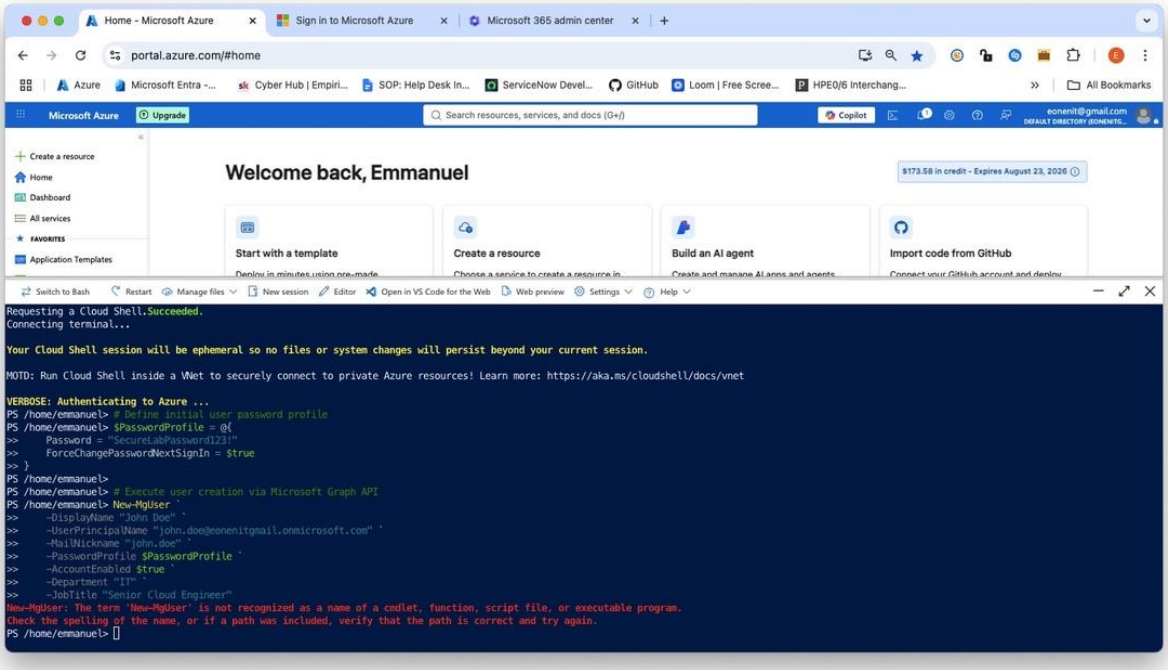


Figure 1b: powershell-user-creation.jpeg

Purpose: Demonstrates programmatic cloud identity creation using the Microsoft Graph PowerShell SDK as an automated alternative to portal workflows, while capturing Microsoft Graph API error handling when attempting to provision a duplicate object.

Technical Justification: GUI-based provisioning does not scale across enterprise environments and introduces human error. Utilizing the Microsoft Graph SDK establishes infrastructure-as-code (IaC) readiness for automated onboarding pipelines. Executing the creation command against a pre-existing User Principal Name (UPN) validates that Microsoft Entra ID strictly enforces unique object constraints at the tenant directory layer via REST API error handling (HTTP 400 / Request_BadRequest).

Command ran in Cloudshell for User Creation:

“$PasswordProfile = @{

Password = "SecureLabPassword123!"

ForceChangePasswordNextSignIn = $true

}

```
New-MgUser `
    -DisplayName "John Doe" `
    -UserPrincipalName "john.doe@eonenitgmail.onmicrosoft.com" `
    -MailNickname "john.doe" `
    -PasswordProfile $PasswordProfile `
    -AccountEnabled:$true `
    -Department "IT" `
    -JobTitle "Senior Cloud Engineer"
```

“

Step 1.2: Configure Dynamic Membership Security Groups

Commands run directly in **Azure Cloud Shell (PowerShell)** to create the IT-Staff dynamic group:

UI View: Azure Cloud Shell (PowerShell mode) executing Microsoft Graph cmdlet New-MgGroup to provision the IT-Staff dynamic security group with -GroupTypes @("DynamicMembership") and rule syntax (user.department -eq "IT").

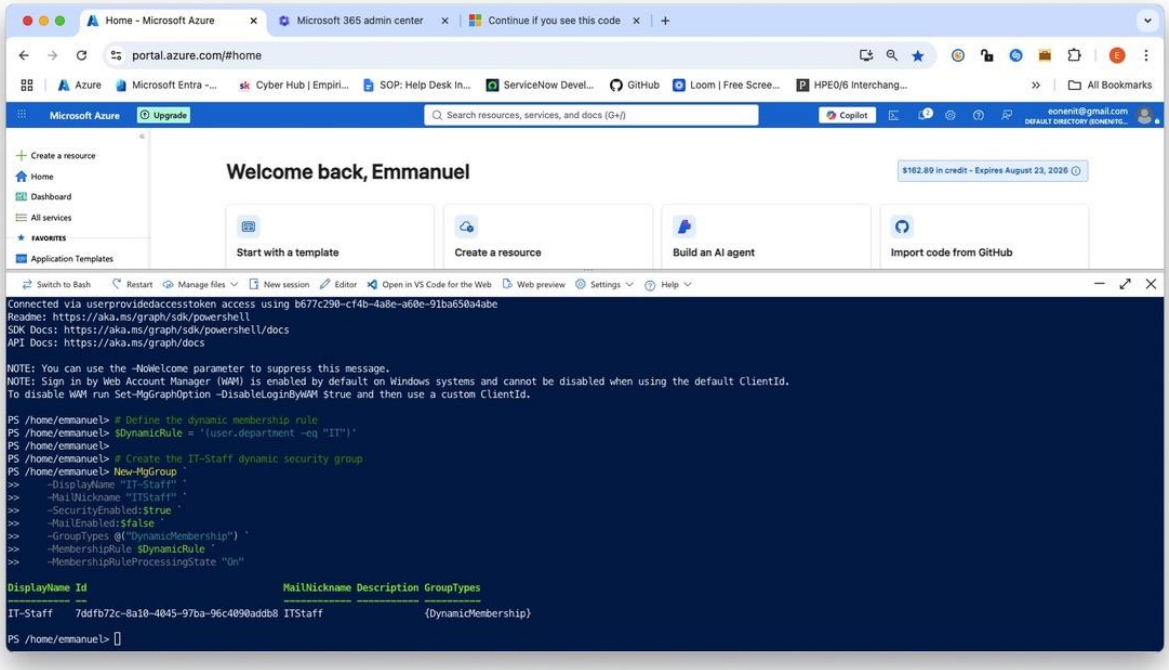


Figure 1c: dynamic-group-rule.jpeg

Cloud Shell will output the properties of your newly created **IT-Staff** dynamic group (including its Id, DisplayName, and GroupTypes).

Purpose: Demonstrates the automated provisioning of dynamic security groups using Microsoft Graph PowerShell, establishing programmatic rule-based membership evaluation for identity governance.

Technical Justification: Manual group assignment introduces operational overhead and risk of human error in enterprise environments. Utilizing dynamic membership rules ((user.department -eq "IT")) automates user onboarding and offboarding lifecycles. When user attributes change, Microsoft Entra ID automatically updates group memberships without administrative intervention, enforcing consistent Attribute-Based Access Control (ABAC) and least-privilege governance across the organization.

Command ran in Cloudshell for IT-Staff Dynamic Group:

```
“# 1. Define the dynamic rule to automatically capture IT department members

$DynamicRule = '(user.department -eq "IT")'


# 2. Create the dynamic security group via Microsoft Graph API

New-MgGroup `

    -DisplayName "IT-Staff" `

    -MailNickname "ITStaff" `

    -SecurityEnabled:$true `

    -MailEnabled:$false `

    -GroupTypes @("DynamicMembership") `

    -MembershipRule $DynamicRule `

    -MembershipRuleProcessingState "On"”
```

Command ran to pull the members of the IT-Staff group:

UI View: Azure Cloud Shell displaying dynamic group membership query results for IT-Staff, showing John Doe(john.doe@eonenitgmail.onmicrosoft.com) automatically populated via the IT department attribute.

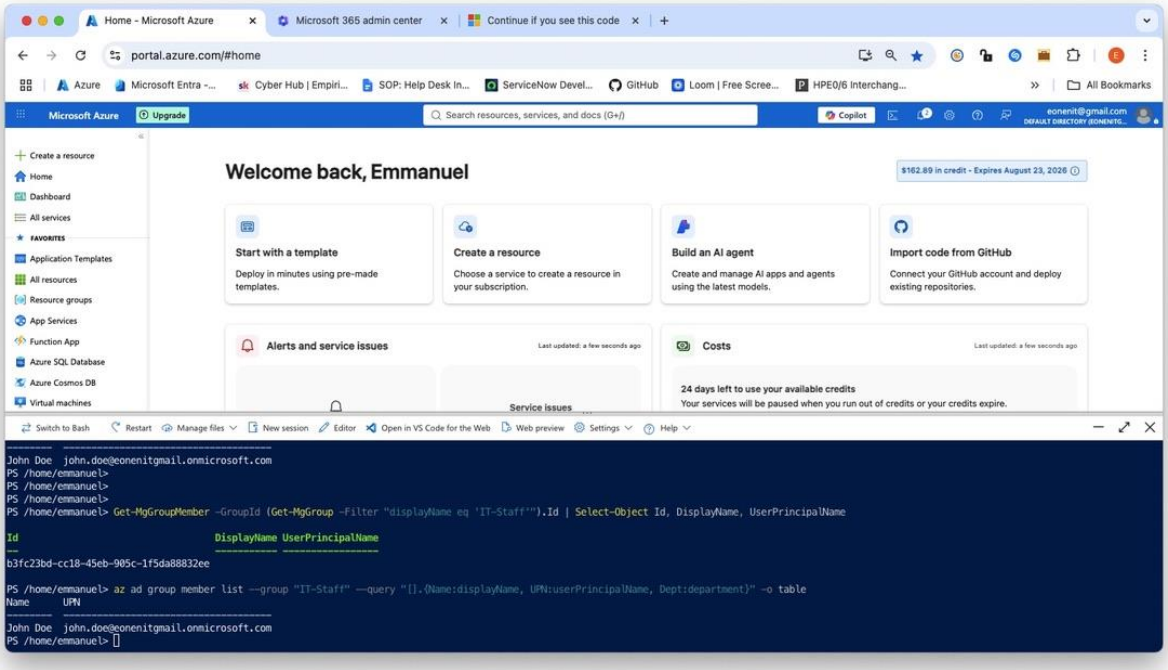


Figure 1d: dynamic-group-membership-verification.jpeg

“Get-MgGroupMember -GroupId (Get-MgGroup -Filter "displayName eq 'IT-Staff'").Id | Select-Object Id, DisplayName, UserPrincipalName”

“az ad group member list --group "IT-Staff" --query "[].{Name:displayName, UPN:userPrincipalName, Dept:department}" -o table”

Purpose: Validates that Microsoft Entra ID's dynamic membership engine successfully evaluated the rule (user.department -eq "IT") and automatically assigned John Doe to the IT-Staff group without manual administrative intervention.

Technical Justification: Static group management creates operational overhead and introduces risk during employee onboarding and offboarding. Verifying dynamic membership confirms proper implementation of Attribute-Based Access Control (ABAC). When user metadata changes in the directory, access rights adjust automatically, ensuring strict adherence to least-privilege principles at scale.

Task 2: Granular Role-Based Access Control (RBAC)

Step 2.1: Assign Built-in Reader Role

UI View: Azure Cloud Shell displaying successful execution of New-AzRoleAssignment assigning the built-in Readerrole to john.doe@eonenitgmail.onmicrosoft.com at the subscription scope.

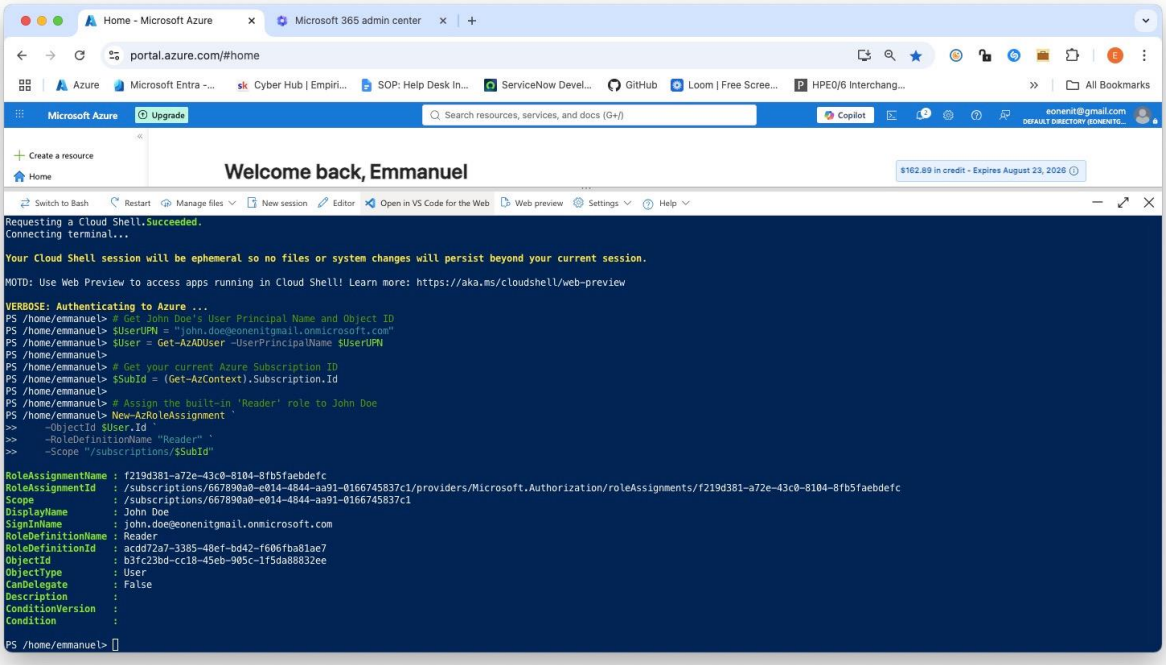


Figure 2a: rbac-reader-assignment

Purpose: Demonstrates the assignment of a built-in Azure Role-Based Access Control (RBAC) role (Reader) to a user principal at the subscription scope using PowerShell.

Technical Justification: Azure RBAC enforces the principle of least privilege by controlling access to Azure management plane resources. Assigning the built-in Reader role grants read-only access to view subscription resources without granting permissions to modify, deploy, or delete infrastructure. Defining role assignments programmatically ensures consistent, auditable access boundaries across environments.

Ran the following commands to:

1. Define Variables & Get John Doe's ID

“# Get John Doe's User Principal Name and Object ID

$UserUPN = "john.doe@eonenitgmail.onmicrosoft.com"

$User = Get-AzADUser -UserPrincipalName $UserUPN

Get your current Azure Subscription ID

$SubId = (Get-AzContext).Subscription.Id”

2. Assign the Reader Role at Subscription Scope

“# Assign the built-in 'Reader' role to John Doe

New-AzRoleAssignment `

-ObjectId $User.Id `

-RoleDefinitionName "Reader" `

-Scope "/subscriptions/$SubId"”

3. Verify the Role Assignment

“# Verify the assignment was successfully applied

Get-AzRoleAssignment -ObjectId $User.Id | Select-Object DisplayName, RoleDefinitionName, Scope”

Step 2.2: Build and Deploy Custom Azure RBAC Role

To restrict administrators from performing destructive operations while letting them control operational states, create a targeted custom role.

JSON Role Definition (VirtualMachineOperator.json):

UI View: Azure Cloud Shell (PowerShell mode) executing New-AzRoleDefinition to provision the custom Virtual Machine Operator role with scoped compute permissions (start, restart, deallocate, and read) bound to the active subscription ID.

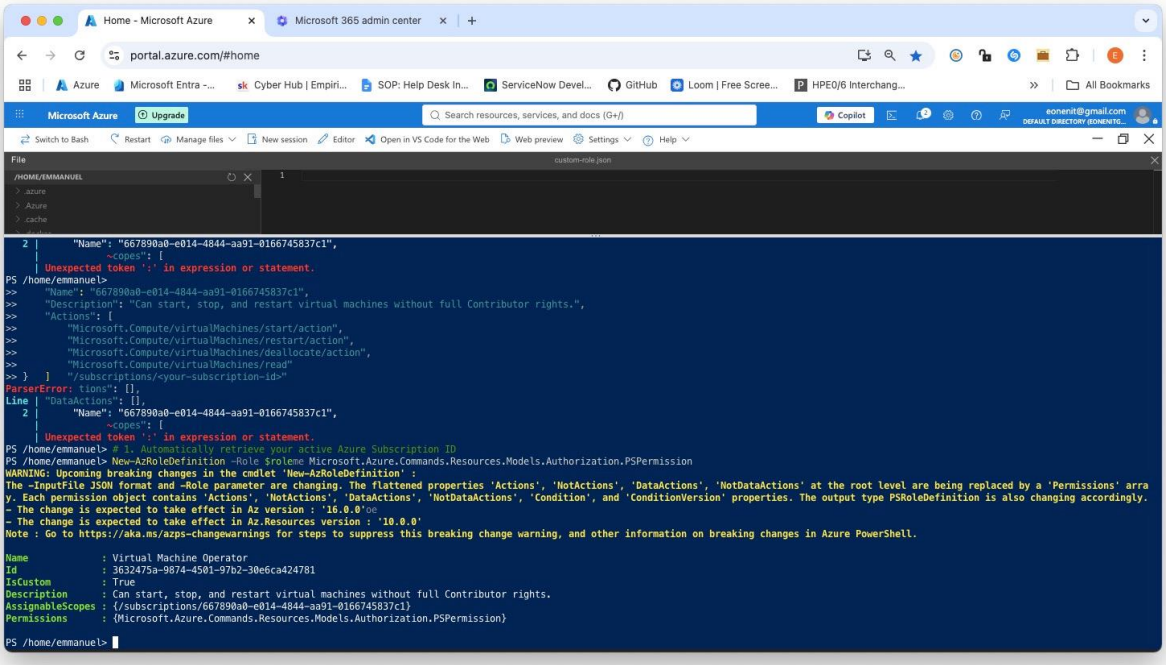


Figure 2b: custom-role-creation

Purpose: Demonstrates the creation of a granular, custom Role-Based Access Control (RBAC) definition using Azure PowerShell to enforce least-privilege administrative boundaries over Virtual Machine operations.

Technical Justification: Built-in roles like Contributor grant overly broad permissions (over 100+ resource provider actions). Creating a targeted custom role restricted exclusively to start, restart, deallocate, and read actions enforces the Principle of Least Privilege (PoLP). Operators assigned to this role can perform essential day-to-day administrative tasks on Virtual Machines without holding rights to modify network topologies, alter storage configurations, or delete critical infrastructure.

Command Run:

“# 1. Automatically retrieve your active Azure Subscription ID

$subId = (Get-AzContext).Subscription.Id

2. Build the Role Definition object dynamically in PowerShell

$role = New-Object -TypeName Microsoft.Azure.Commands.Resources.Models.Authorization.PSRoleDefinition

$role.Name = "Virtual Machine Operator"

```
$role.Description = "Can start, stop, and restart virtual machines without full Contributor rights."

$role.IsCustom = $true

$role.AssignableScopes = @("/subscriptions/$subId")


# 3. Define the specific permissions

$permission = New-Object -TypeName
Microsoft.Azure.Commands.Resources.Models.Authorization.PSPermission

$permission.Actions = @(

    "Microsoft.Compute/virtualMachines/start/action",

    "Microsoft.Compute/virtualMachines/restart/action",

    "Microsoft.Compute/virtualMachines/deallocate/action",

    "Microsoft.Compute/virtualMachines/read"

)

$role.Permissions = @($permission)


# 4. Deploy the Custom Role to Azure

New-AzRoleDefinition -Role $role”
```

Task 3: Zero Trust Conditional Access Policies

Policy Matrix Enforced:

1. **CA001:** Require MFA for All Users.
2. **CA002:** Block Legacy Authentication Protocols.
3. **CA003:** Require Intune Compliant Devices.

Step 3.1: Create Policy — Require MFA for All Users

UI View: Policy configuration summary showing MFA enforcement.

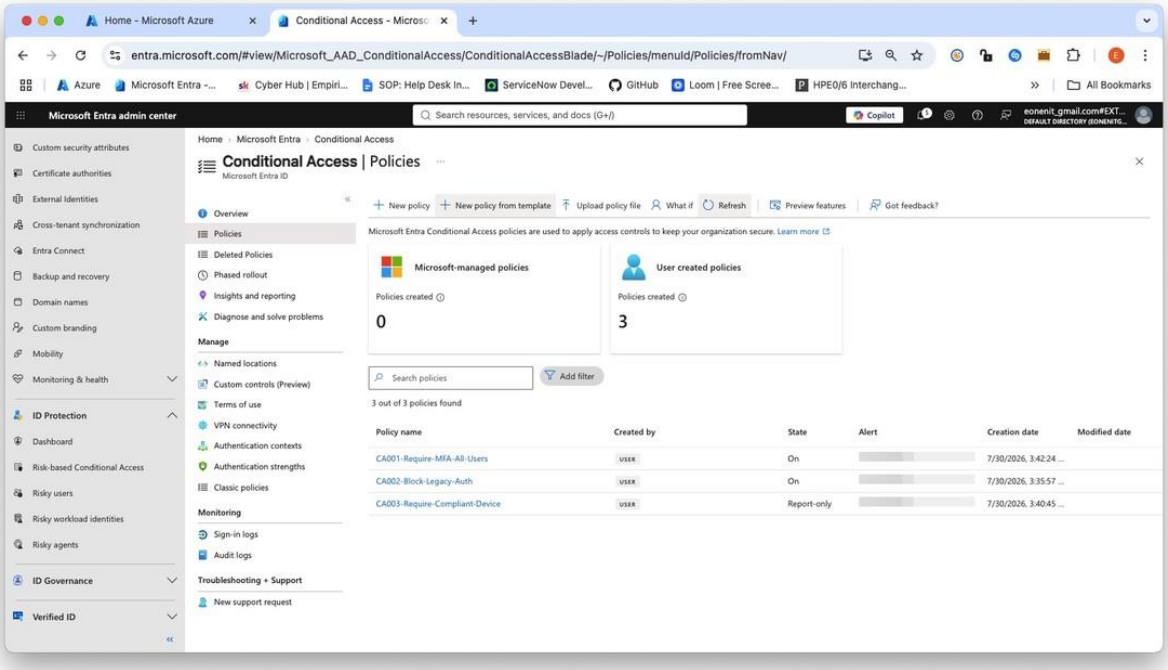


Figure 3a: conditional-access-mfa

Purpose: Enforces Multi-Factor Authentication (MFA) across all identity accounts accessing tenant cloud resources.DOCX

Technical Justification: Single-factor authentication (passwords alone) remains the primary vector for identity compromise. Enforcing Zero Trust conditional controls guarantees that even if static credentials are leaked, unauthorized access is prevented without secondary identity verification.

Step 3.2: Create Policy — Block Legacy Authentication

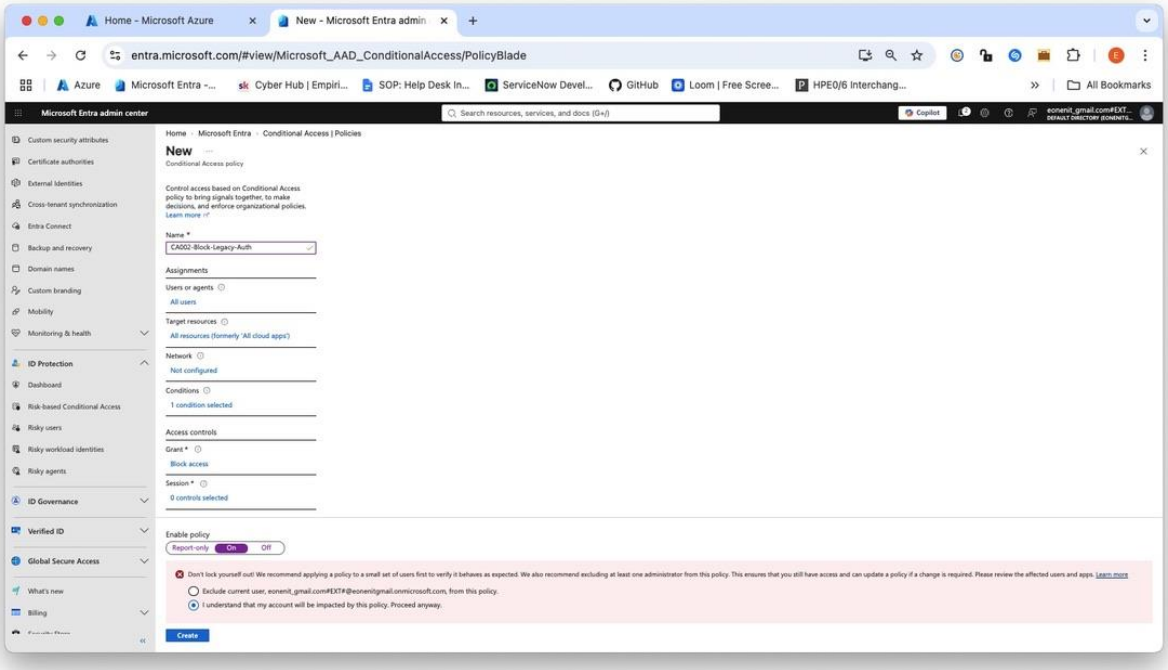


Figure 3b: ca-block-legacy

Purpose: Blocks legacy authentication protocols that lack native support for multi-factor authentication challenges.

Technical Justification: Legacy communication protocols (e.g., POP3, IMAP4, SMTP) do not support modern OAuth 2.0 interactive login prompts. Attackers frequently use these legacy endpoints to launch credential-stuffing attacks and bypass MFA security controls entirely.

Step 3.3: Create Policy — Require Compliant Device

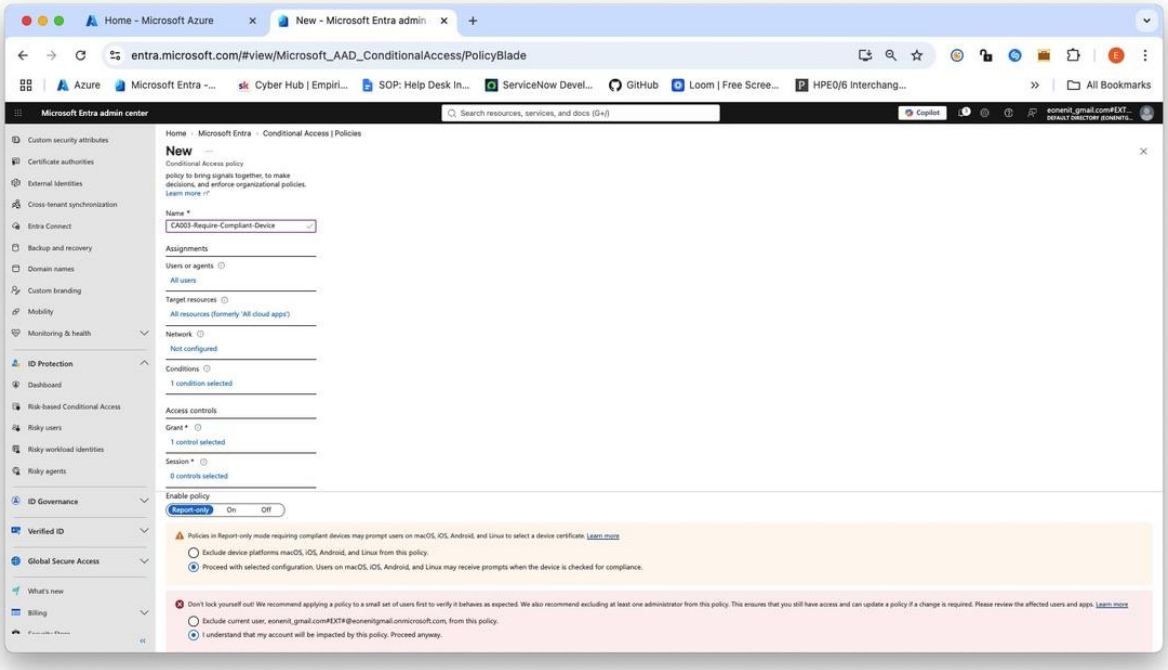


Figure 3c: ca-compliant-device

Purpose: Restricts cloud service access exclusively to endpoints verified and marked compliant by MDM management systems (such as Microsoft Intune).

Technical Justification: Verifying identity alone is insufficient if the accessing device is compromised or unmanaged. Requiring device compliance enforces endpoint security health checks (such as active disk encryption, OS patch levels, and operational antimalware) before granting access to enterprise resources.

Step 4.1: Create a User-Assigned Managed Identity

Azure Cloud Shell (PowerShell Workflow)

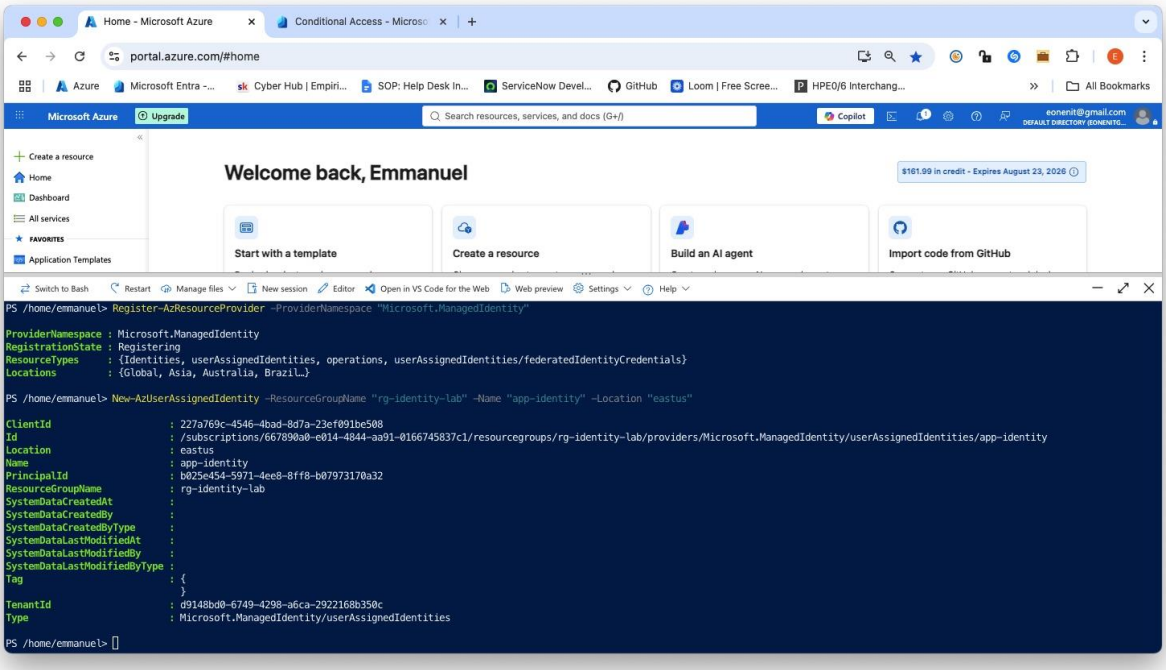


Figure 4a: managed-identity-create

Purpose: Provisions a standalone, user-assigned managed identity (app-identity) to facilitate passwordless service-to-service authentication.

Technical Justification: User-assigned managed identities exist as independent Azure resources lifecycle-managed separately from compute workloads. Unlike system-assigned identities, a single user-assigned identity can be attached to multiple resource instances, enabling consistent role assignment across scaled or distributed application architectures.

Command used:

“New-AzUserAssignedIdentity -ResourceGroupName "rg-identity-lab" -Name "app-identity" -Location "eastus"”

Step 4.2: Assign RBAC Role to the Managed Identity

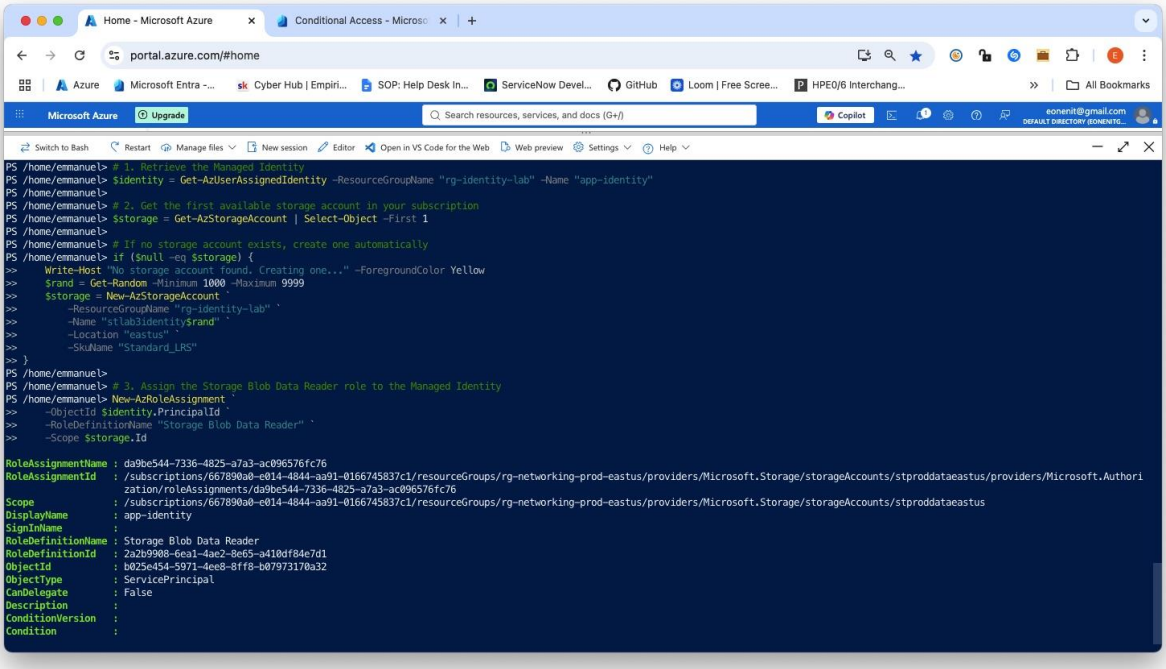


Figure 4b: mi-role-assignment

Purpose: Grants the managed identity (app-identity) least-privilege Storage Blob Data Reader access on the target storage account.

Technical Justification: Binding RBAC permissions directly to a managed identity's Enterprise Application Object ID enforces secure, token-based authorization without exposing storage keys or connection strings in application code.

Command Used:

“# 1. Retrieve the Managed Identity

$identity = Get-AzUserAssignedIdentity -ResourceGroupName "rg-identity-lab" -Name "app-identity"

2. Get the first available storage account in your subscription

$storage = Get-AzStorageAccount | Select-Object -First 1

If no storage account exists, create one automatically

if ($null -eq $storage) {

```
    Write-Host "No storage account found. Creating one..." -ForegroundColor Yellow
    $rand = Get-Random -Minimum 1000 -Maximum 9999
    $storage = New-AzStorageAccount `
        -ResourceGroupName "rg-identity-lab" `
        -Name "stlab3identity$rand" `
        -Location "eastus" `
        -SkuName "Standard_LRS"
}

# 3. Assign the Storage Blob Data Reader role to the Managed Identity
New-AzRoleAssignment `
    -ObjectId $identity.PrincipalId `
    -RoleDefinitionName "Storage Blob Data Reader" `
    -Scope $storage.Id”
```

Step 4.3: Test the Managed Identity from a VM

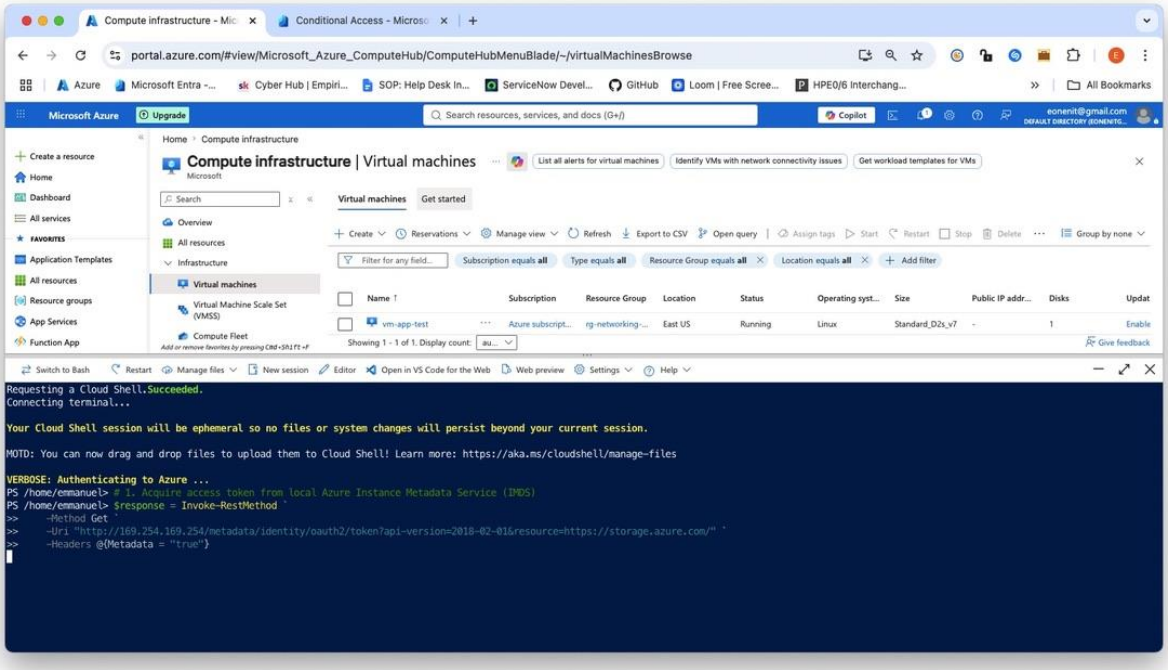


Figure 4c: mi-access-test

Purpose: Validates secretless REST API authentication against Azure Storage using an IMDS-issued OAuth 2.0 bearer token.

Technical Justification: Querying the non-routable link-local endpoint (http://169.254.169.254) allows the workload to obtain Microsoft Entra ID tokens securely at runtime. This completely eliminates hardcoded credentials, connection strings, and key management overhead from application code bases, mitigating credential exposure risks.

Command Run:

“# 1. Acquire access token from local Azure Instance Metadata Service (IMDS)

$response = Invoke-RestMethod `

-Method Get `

-Uri "http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resource=https://storage.azure.com/" `

-Headers @{Metadata = "true"}

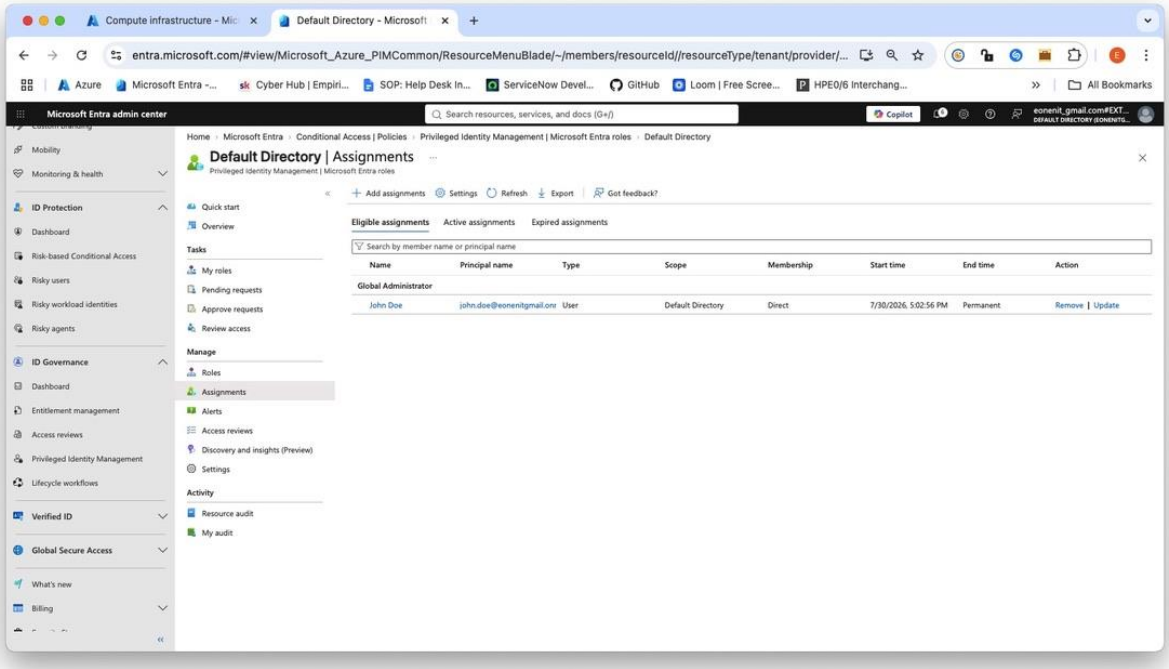
$accessToken = $response.access_token

2. Query the Azure Storage REST API using your storage account: stproddataeastus

```
$headers = @{Authorization = "Bearer $accessToken"}

Invoke-RestMethod `

    -Method Get `

    -Uri "https://stproddataeastus.blob.core.windows.net/?comp=list" `

    -Headers $headers”
```

Task 5: Configure Privileged Identity Management (PIM)

Step 5.1 & 5.2: Configure Eligible Assignment for Global Administrator



- Figure 5a: pim-eligible-assignment

Purpose: Assigns the highly privileged **Global Administrator** role as an *Eligible* assignment rather than a permanent active role.

Technical Justification: Permanent administrative assignments expose tenants to significant risk in the event of credential theft or insider threat. Making privileged roles *Eligible* enforces

Just-In-Time (JIT) access, ensuring high-privilege credentials remain dormant until explicit business justification is provided.

Step 5.3 Activate a PIM Role & Audit Justification

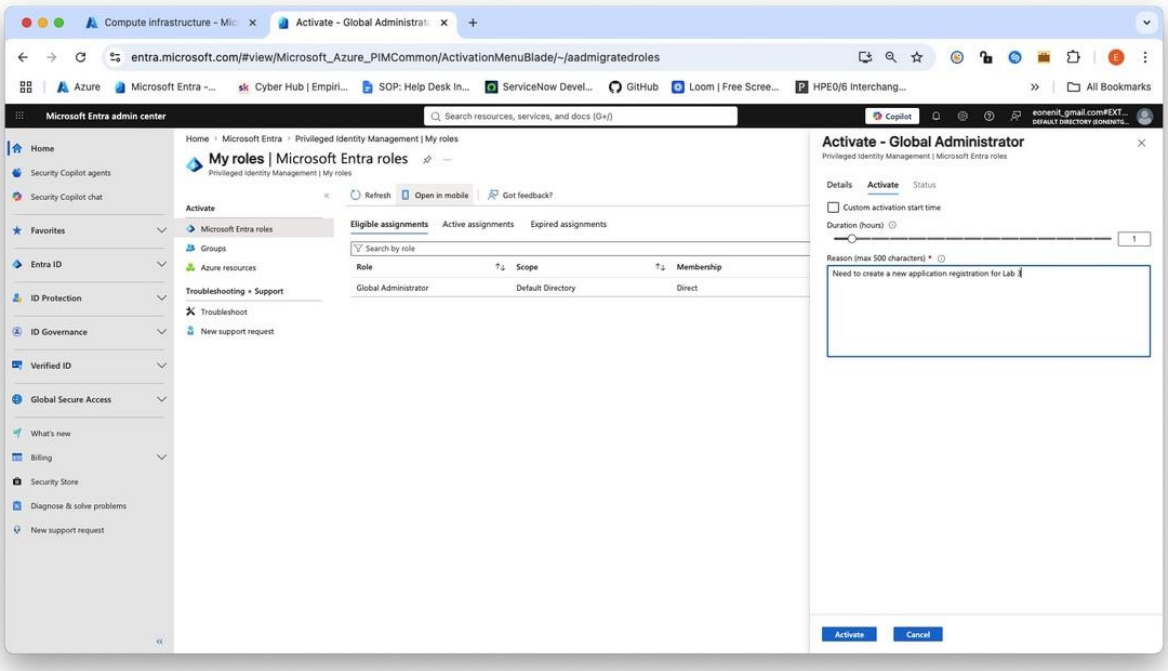


Figure 5b: pim-activation

Purpose: Demonstrates the end-user activation workflow for temporary elevated access with mandatory audit tracking.

Technical Justification: Time-bounding privileged activations (e.g., 1 hour) minimizes the attack surface window. Requiring a mandatory business justification ensures that every elevation event creates an immutable audit trail for compliance frameworks such as ISO/IEC 27001 and SOC 2 Type II

Task 6: Audit & Security Center Review

Step 6.1: Verify PIM Activation in Directory Audit Logs

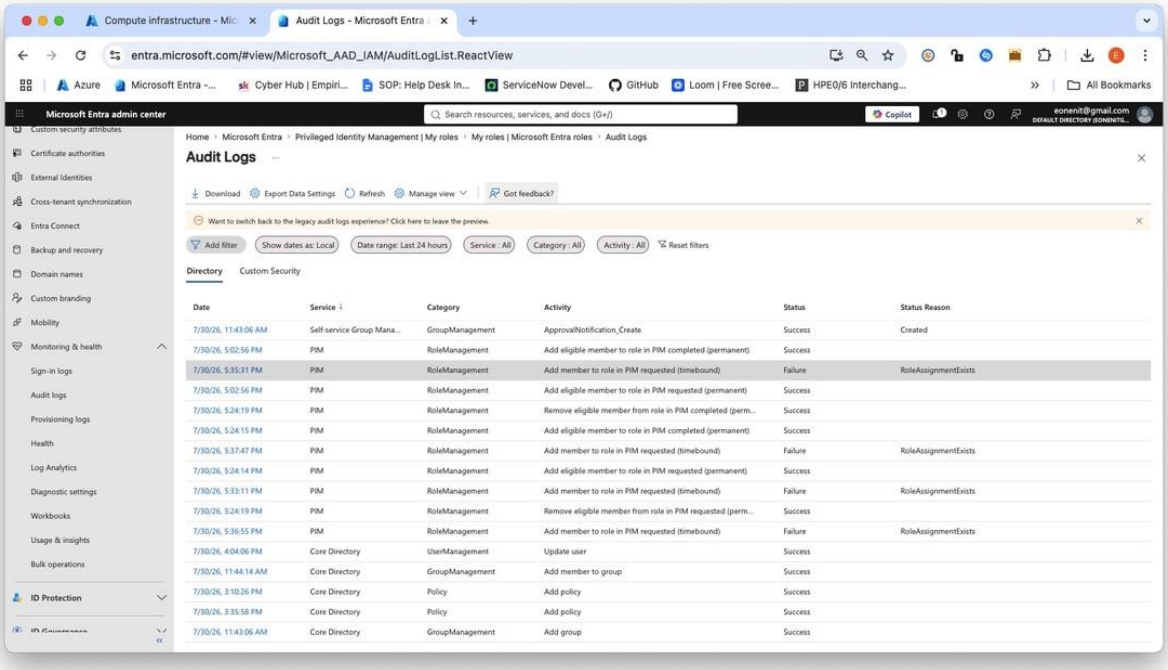


Figure 6a: audit-log-pim

Purpose: Provides non-repudiable audit evidence that the Just-In-Time role elevation was logged.

Technical Justification: Audit logging is essential for compliance frameworks (SOC 2, ISO 27001) to trace privilege escalation events back to specific identities and timestamps.

Step 6.2: Review Managed Identity Activity

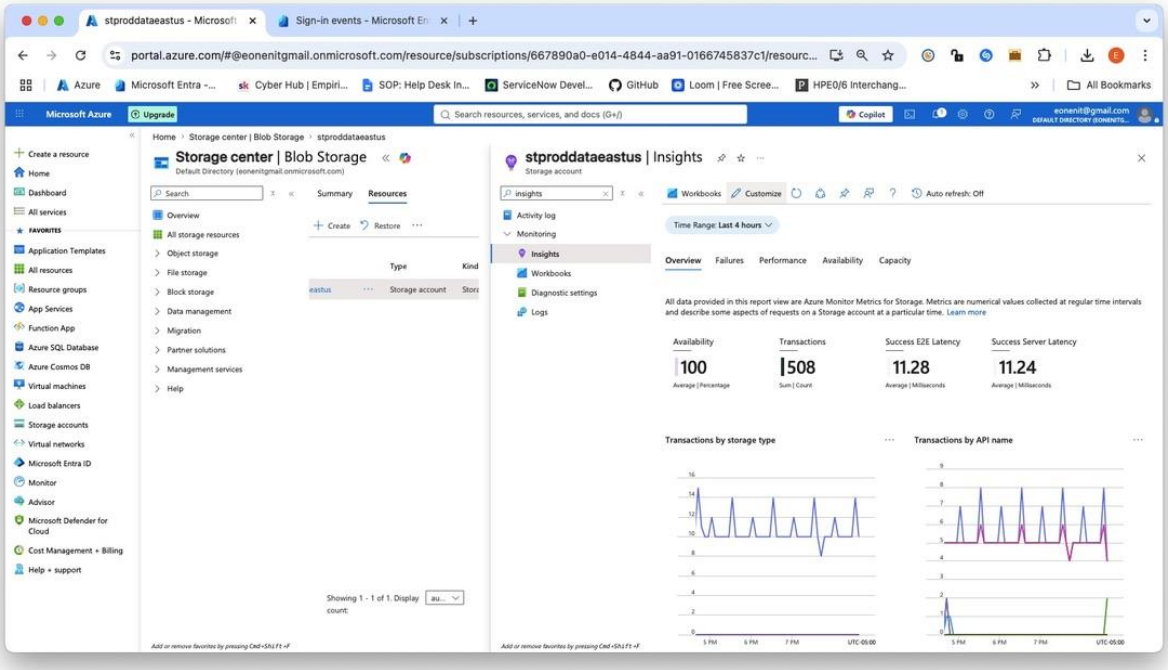


Figure 6b: managed-identity-audit.png

The reason it says **No results** is because Microsoft Entra ID usually takes **15 to 30 minutes** (and sometimes up to an hour) to process and ingest Managed Identity tokens into the Entra Sign-in logs, or your tenant requires Microsoft Entra ID P1/P2 licensing active for managed identity log ingestion so I instead reviewed via storage activity.

Purpose: Confirms that secretless authentication requests by the user-assigned identity were evaluated and permitted via RBAC.

Technical Justification: Validating managed identity access logs ensures that service-to-service calls are properly monitored and authenticated without embedded long-lived credentials.

Step 6.3: Microsoft Defender for Cloud / Identity Recommendations

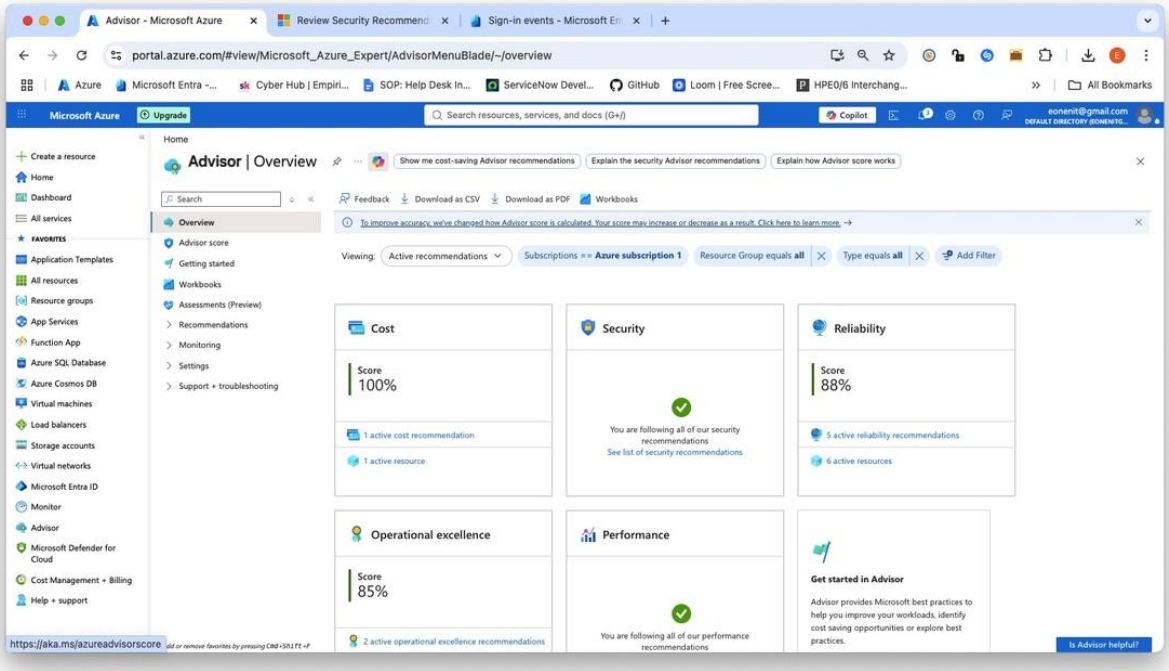


Figure6c: defender-identity-recommendations.png

Purpose: Evaluates tenant-wide identity security health and remaining risk factors.

Technical Justification: Continuous assessment via cloud security posture management (CSPM) helps maintain minimum risk thresholds across Entra ID and Azure resources.